

Forum public & actualités tech — comment c'est pensé

Horus-Lab · briefing technique · 28 juillet 2026 · branche `feat/refonte-site-pro`

Deux fonctionnalités ont été ajoutées sous les articles du blog : un **forum public sans inscription** et une section « **Actualités tech** » **alimentée automatiquement**. Ce document explique le raisonnement derrière chaque choix — pas seulement ce qui a été codé, mais *pourquoi* c'est codé ainsi.

1. Le forum — un fil de discussion par article

La contrainte de départ

On voulait que n'importe quel lecteur puisse réagir à un article **immédiatement**. Or toute barrière (créer un compte, valider un e-mail) fait chuter la participation de façon spectaculaire sur un site qui démarre. Décision : **zéro inscription**. Mais « zéro inscription » ouvre la porte au spam — c'est donc toute l'architecture qui devait compenser cette ouverture.

La structure des données

```
ForumThread (1 fil par article, identifie par le "slug" de l'article)
├── ForumPost (un message public)
│   ├── reply_to → ForumPost « qui répond à qui » (NOUVEAU)
│   ├── is_staff → badge « Equipe Horus-Lab »
│   ├── is_hidden → moderation : disparaît du site, reste en base
│   └── author_email → STOCKE mais JAMAIS expose par l'API
```

Le fil est créé **à la volée** au premier message : aucune action n'est requise dans l'admin pour « ouvrir » la discussion d'un article. Le lien se fait par le `slug`, donc un article renommé dans le CMS garde son fil tant que son slug ne change pas.

Les six décisions, et leur raison

Décision	Pourquoi
Deux serializers séparés (lecture / écriture)	Le serializer de lecture ne contient physiquement pas le champ e-mail. Une fuite devient impossible par construction, pas par vigilance.
Champ piège invisible (<code>website</code>)	Les robots remplissent tous les champs d'un formulaire. Un humain ne voit pas celui-ci. Filtre gratuit, sans CAPTCHA — donc sans friction pour le lecteur.
Limite de débit à l'écriture seule (20 messages/h/IP)	La lecture reste totalement libre (elle est sollicitée toutes les 10 s par le sondage). Seule la publication est plafonnée.
Masquer plutôt que supprimer	Un message modéré disparaît du site mais reste consultable en base : on peut revenir sur une décision, et on garde une trace en cas de litige.
Proxy Next.js <code>/api/forum/[slug]</code>	Le navigateur ne connaît jamais l'URL du backend Django. Les champs sont aussi tronqués côté proxy avant d'atteindre l'API (nom 120, texte 5 000...).
Notifications en tâche de fond	E-mail (Brevo) et/ou Telegram partent dans un fil séparé, et toute erreur est avalée : une notification ratée ne doit jamais faire échouer la publication d'un message .

Ce que voit le visiteur

Le message s'affiche **instantanément**, avant même la réponse du serveur (affichage optimiste), puis se consolide quand l'API confirme. Si l'envoi échoue, la bulle passe en rouge avec « Non publié, réessayer » plutôt que de disparaître — on ne perd jamais le texte du lecteur. Toutes les 10 secondes, le navigateur demande uniquement les messages **postérieurs au dernier identifiant connu** (`?after=<id>`) : la conversation vit en direct sans jamais recharger tout le fil.

Nouveauté : « qui répond à qui »

Un bouton **↩ Répondre** sous chaque message cite l'auteur et un extrait (140 caractères) au-dessus de la réponse. Trois garde-fous : la citation ne peut viser qu'un message **du même fil** et **non masqué** (vérifié côté serveur, pas seulement dans le navigateur) ; si le message cité est supprimé, la réponse **survit** et perd simplement sa citation. Côté admin, l'équipe peut elle aussi répondre à un message précis, et le fil affiche visuellement les citations.

2. Actualités tech — pourquoi ce n'est *pas* du scraping

Le mot « scraping » désigne l'aspiration du HTML d'un site pour en extraire le contenu. C'est **fragile** (le moindre changement de design casse tout), **juridiquement risqué** (reproduction d'œuvres protégées) et souvent **contraire aux conditions d'utilisation** des médias. Nous avons donc pris le chemin inverse : les **flux RSS**, que ces mêmes médias publient *exactement pour cet usage*.

La règle que l'on s'impose

On ne stocke que **le titre, un résumé, l'image d'illustration et le lien. Jamais l'article entier**. Chaque carte du site renvoie vers la source (« Lire sur The Verge »), qui garde son trafic et ses revenus. C'est le fonctionnement normal d'un agrégateur, et cela nous met à l'abri sur le plan du droit d'auteur.

Le pipeline, étape par étape



Étape	Ce qui se passe
Les sources	7 médias 100 % tech , aucune source généraliste : TechCrunch, The Verge, Ars Technica, Wired, Hacker News (<i>mondial</i>) + TechCabal, Disrupt Africa (<i>écosystème tech africain</i>). Ce dosage colle au positionnement d'Horus-Lab.
Le dédoublement	L'URL de l'article est la clé unique en base. Un même sujet repris par un flux dix fois n'entre qu'une seule fois — et surtout, un article déjà connu n'est pas retraduit .
La traduction	Les flux sont anglophones. L'anglais est conservé tel quel (c'est l'original), le français est produit par l'API gratuite MyMemory. En cas de quota atteint ou de panne, on retombe sur le texte anglais : une brève en anglais vaut mieux qu'une brève absente.
La purge	6 articles récents par flux à chaque passage, 42 conservés en base au total, les plus anciens sont supprimés. La base ne grossit jamais indéfiniment.
La fréquence	Une tâche planifiée (cron) sur le serveur, toutes les 4 heures . C'est le rythme de publication réel de ces médias ; plus souvent ne servirait à rien.

Le principe de dégradation

Chaque maillon peut tomber sans casser le site. Un flux injoignable ? Les six autres continuent. L'API de traduction indisponible ? Texte anglais. Le backend Django éteint ? La page blog renvoie une liste vide et **la section disparaît purement et simplement** — pas de bloc vide, pas d'erreur, le lecteur ne voit rien d'anormal. Ce n'est pas un détail : c'est la raison pour laquelle une fonctionnalité « bonus » ne peut jamais faire tomber la page blog.

Contrôle éditorial

Chaque brève est visible dans l'admin Django sous « **Actualités tech (auto)** », avec une case **Actif** décochable directement depuis la liste. Une brève hors sujet ou mal traduite se retire en un clic, sans toucher au code. La collecte reste automatique, mais **le dernier mot reste éditorial**.

État actuel

FAIT 41 brèves collectées et traduites en local, section visible sur `/fr/blog` et `/en/blog`. **À FAIRE** en production : lancer la commande une première fois puis installer le cron — la procédure exacte est dans `RESTE-A-FAIRE.md`, étape 7. Sans cela, la section restera simplement invisible en ligne.

3. Lecture des articles — ce qui vient d'être ajouté

Le **titre est désormais justifié** comme le corps du texte (la dernière ligne reste alignée à gauche, ce qui évite les espaces étirés). Surtout, le corps de l'article dispose d'un **défilement interne automatique** : sa hauteur est plafonnée à `clamp(26rem, 74svh, 60rem)` — jamais moins de 416 px sur un petit mobile, environ trois quarts de l'écran en usage normal, jamais plus de 960 px sur un grand moniteur. Un article court ne déclenche **rien du tout** ; un article très long devient défilant dans son cadre, avec un dégradé en bas qui signale qu'il reste du texte, et le forum reste accessible sans faire défiler la page pendant dix secondes.

Trois points de finition : le plafonnement est en **CSS pur** (il fonctionne même si le JavaScript ne se charge pas), la zone est **focusable au clavier** pour être parcourue aux flèches sans souris, et le « scroll chaining » est désactivé — au doigt sur mobile, atteindre le bas du cadre ne relance pas brutalement le défilement de la page.

4. Améliorations que je recommande

Priorité	Sujet	Pourquoi / quoi
Haute	Cron des actualités en prod	Sans lui, la section « En direct » restera vide en ligne alors qu'elle fonctionne en local. C'est le seul écart bloquant entre le local et la production aujourd'hui.
Haute	Modération du forum : e-mail à l'équipe	Le code de notification existe et est prêt, mais il faut renseigner <code>BREVO_API_KEY</code> (et éventuellement Telegram) dans le <code>.env</code> du serveur. Sans cela, personne ne saura qu'un message a été posté — et un forum sans réponse fait plus de mal que pas de forum.
Haute	Filtre anti-injures minimal	Un forum sans inscription finira par recevoir du spam malgré le champ piège. Une liste de mots interdits qui met automatiquement le message en « masqué » (au lieu de le rejeter) coûte peu et évite de découvrir l'insulte trois jours plus tard.
Moyenne	Images distantes cassées	Si l'illustration d'une brève RSS disparaît chez la source, il reste un rectangle gris. Il faut passer la carte en composant client pour masquer l'image en cas d'échec de chargement.
Moyenne	Fil de discussion imbriqué	Aujourd'hui les réponses restent chronologiques avec une citation. Au-delà de ~30 messages, un véritable regroupement par conversation deviendra plus lisible. Le modèle de données le permet déjà : c'est uniquement de l'affichage.
Moyenne	Sommaire d'article	Puisque les articles longs défilent maintenant dans un cadre, un sommaire cliquable des titres H2/H3 à côté du texte rendrait la navigation bien plus confortable sur ordinateur.
Basse	Avertissement de build Turbopack	Un import dans <code>sitemap.ts</code> fait analyser tout le projet au moment du build, ce qui alourdit l'image Docker. Sans impact visible, mais à nettoyer un jour.
Basse	Qualité de la traduction	MyMemory est gratuit mais parfois approximatif sur le jargon technique. Si le besoin se confirme, DeepL propose 500 000 caractères/mois gratuits, avec un rendu nettement meilleur. Le code est déjà isolé dans une seule fonction : le changement serait localisé.

En résumé

Le fil conducteur des deux fonctionnalités est le même : **ne jamais laisser une fonctionnalité secondaire mettre en péril l'essentiel**. Le forum accepte tout le monde mais filtre à cinq endroits différents ; les actualités s'alimentent seules mais restent débrayables en un clic et disparaissent silencieusement en cas de panne. Ce sont ces garde-fous, plus que les fonctionnalités elles-mêmes, qui permettent de mettre le site en ligne sereinement.